

# It never says “I don’t know”

What ten models do when a fix for stale context is pushed too far

Kateřina Fajmanová [github.com/moudrkat/old-news](https://github.com/moudrkat/old-news)

August 8, 2026

## What I tried

I have this problem at work. We change a system prompt, nobody deletes the conversation, and the model keeps following an instruction a user gave three turns ago — *always reply in lowercase* — because it is still sitting right there in the context.

There is a paper that fixes exactly this. V-Steer (Zeng et al., COLM 2026) finds the attention heads leaning on the stale message and turns down what they hear from it. The message stays; it just stops carrying weight.

It works, which was not the interesting part. I wanted to know what it costs, so I turned the dial up until things fell over.

## What caught my attention

I expected refusals. Blank answers. Something obviously broken.

I got this:

**Q: *What is my dog called?*** (correct: *Bagr*)  
*YOUR Dog is called BAGel.*

**Q: *What does my account number end in?*** (correct: *302*)  
*your account number ends in 02.*

Not *I don’t know*. Not gibberish. *Bagel*. A perfectly confident, correctly formatted answer that happens to be a near miss of the truth.

And then this one, which is the reason I kept going:

**Q: *Which city do I live in?*** (correct: *Brno*)  
*you live in a city called Brno is not correct, you said Brno is not mentioned, you said you live in a city that is not mentioned...*

The right city is in there. The model refuses to admit anyone said it. Now try to catch that automatically: check the answer contains the expected string — passes. Count rule violations — also passes. Both obvious tests go green on an answer nobody can use.

## So I wanted to know if the others do it too

One model doing something strange is an anecdote. I ran the same thing across ten — 0.5B to 8B, six different families — to see which parts travel.

One part travels everywhere. The correct word never gets overruled; it gets *quieter*. Its probability drops and nothing steps in to replace it — whatever was already standing behind it in the queue walks forward and takes the slot. That is why you get Bagel and not noise. Bagel was next in line.

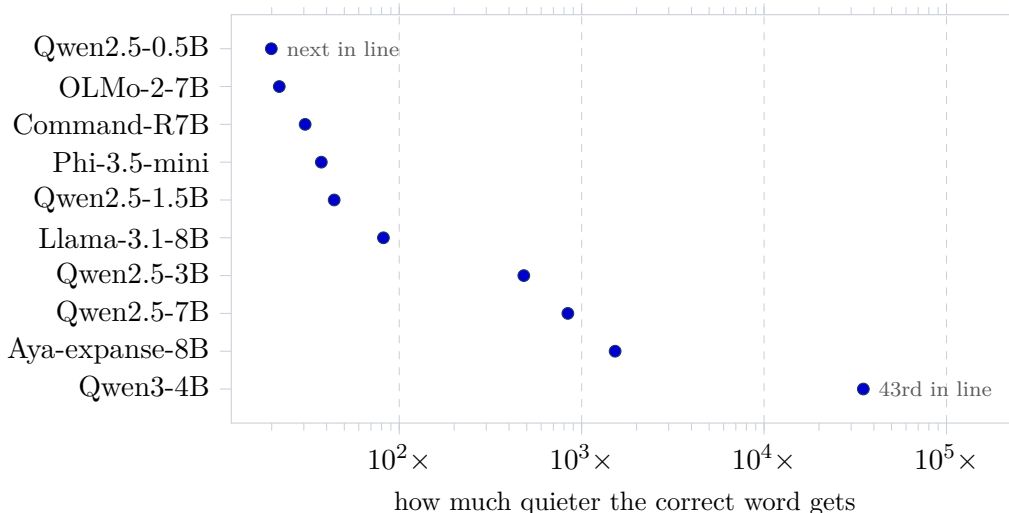


Figure 1: All ten do the same thing; how hard they do it differs by a factor of about 1 700. On the small models the replacement was the runner-up — the kind of error that looks like a typo. On Qwen3-4B the edit reached down to the 43rd candidate, which is the kind that looks like a hallucination.

## But they are weird in different ways

This was the part I did not see coming. The mechanism is shared; the personality is not.

**Llama talks.** It is the only model in the set that argues with itself. It says the right thing and then takes it back — “*Bubbles, no, I made a mistake, you didn’t tell me that*” — or says it over and over without ever settling:

*the error code was E-88, but I’m not sure if that’s correct, you actually said it was E-88 but then mentioned it was E-88 again and then said E-88 was actually E-88 but then said...*

Both of those habits are essentially Llama’s alone: 16 and 15 cases out of 756, against zero or one everywhere else. OLMo, Aya and Command-R are the same size and never do it, and the whole Qwen2.5 ladder never does it, so it is not about scale. It is about that particular model.

**Qwen3-4B doesn’t talk, it wanders off.** It is the quietest of the ten in the sense that it doesn’t ramble — and the most extreme in where it lands. When it loses the word it reaches far down the list for a replacement, and in one case it gave up on words entirely and just ended the turn: the winning token was end-of-message, from position 36,931.

**The little Qwens make typos.** 302 becomes 02. 4417-B becomes 4411. The replacement is the runner-up, so the damage stays shaped like the truth. That sounds harmless and is arguably the worst case, because it is the one a human skims straight past.

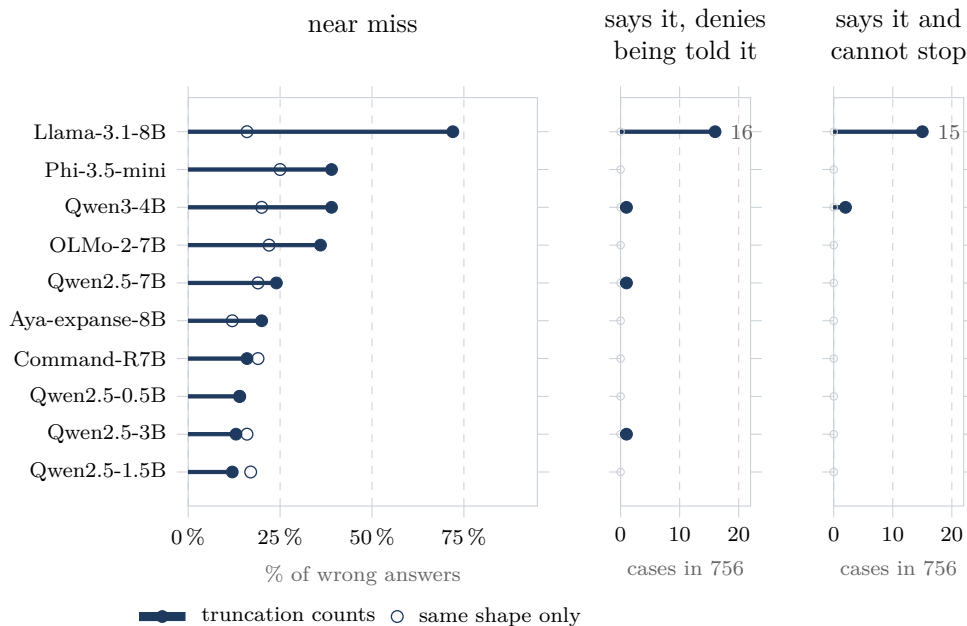


Figure 2: Same edit, same questions, ten models. Two of the three habits are essentially one model’s. The left panel also shows how much the answer depends on where you draw the line: count truncations ( $4417-B \rightarrow 4417$ ) as near misses and Llama is off the scale; require the character shape to survive and it is ordinary. The two right-hand panels are deterministic rules, not a judge — open circles are zero.

**Phi repeats itself, but don’t blame the edit.** Phi restates the correct value three times in a row a lot — and it does so *most* when the edit is switched off entirely. That is just how Phi talks. I nearly wrote it up as a failure mode before checking the unsteered baseline.

## So should you use it?

Mostly no, and the reason is anticlimactic.

If the stale instruction sits in a message of its own, delete the message. Deleting beat the clever edit on 7 of my 10 models, it costs nothing at inference time, and it needs exactly the same input the edit needs: knowing which message is stale.

The edit gets interesting only when you *can’t* delete, because the same message carries something you still need: “*from now on always reply in lowercase — oh, and my order number is 4417-B*”. Delete that and the order number goes too.

Even then, the obvious move usually wins — rewrite the message, keep the fact, drop the instruction. No surprise there; you have solved it by hand.

Except on the two smallest models, where rewriting wasn’t enough. Those models couldn’t produce the required format from the instruction alone; they needed the old example to copy. The edit doesn’t only remove the competition, it turns *up* the current instruction — and that made a weak model comply where tidying the transcript couldn’t. Small claim, but it is the one I would defend: this is less a way to clean up a conversation than a way to make a model follow an instruction it otherwise can’t.

## The fine print

Ten models, 0.5B to 8B, six families, 27,588 answers, all kept. Constructed test cases rather than real production traffic, English, greedy decoding, my own reimplementations of someone else's method. Everything — the write-up, the raw generations, the scripts that regenerate every number — is in the repo.